

STATUS: exploratory

LLM Document Ingestion

Patrick Nichols
zzpanic@gmail.com

· Version v0.1-exploratory

Converting Technical Documents to Machine-Readable Markdown

A Methodology for High-Fidelity Extraction of Structured Content and Mathematical Formulas

Version 1.0 — May 2026

Abstract

Technical documents—engineering standards, academic papers, textbook chapters—contain structured knowledge encoded in complex layouts with mathematical formulas, tables, and dense typographical conventions. Converting these documents into machine-readable markdown format is deceptively challenging: traditional PDF extraction tools produce garbled output with corrupted symbols and broken structure. This whitepaper describes a practical methodology for converting any technical document into clean, searchable markdown with properly formatted mathematical formulas using multimodal AI models. We discuss the problem of technical document digitization, the step-by-step workflow, and quality considerations for preserving mathematical notation and document hierarchy.

1. Introduction

Engineering professionals, researchers, and students routinely consult technical documents that contain hundreds of pages of formulas, design criteria, and procedural guidance. These documents are typically available only as scanned PDFs or proprietary digital formats. The knowledge they contain is trapped in a format that resists computational analysis, semantic search, or automated cross-referencing.

The emergence of multimodal AI models capable of understanding both text and visual layout has raised new possibilities: can we convert scanned technical documents into clean markdown with properly formatted LaTeX formulas? In practice, the answer is yes—but achieving high-quality results requires careful attention to the workflow, from image preparation through prompt design to post-processing.

This document describes a general methodology that anyone can follow, regardless of the specific tools or models they choose, to convert technical documents into machine-readable mark-

down with mathematical fidelity.

2. The Problem of Technical Document Digitization

2.1 Why Traditional PDF Extraction Fails

Standard PDF text extraction tools operate on a simple premise: the PDF contains embedded text strings with positioning metadata. Extract them, reassemble in order, and you have readable content. This works well for documents that were born digital with proper text layers.

Technical engineering standards tell a different story. They are often: - Scanned from physical copies (no text layer) - Typeset with complex mathematical notation rendered as glyph positions - Multi-column layouts that confuse linear extraction - Contain hand-drawn diagrams and proprietary symbols

The result of naive extraction is what practitioners call “garbage text”—blocks of seemingly random characters, corrupted Unicode, and structural chaos. Common symptoms include single-character watermarks (PDF extraction artifacts), mangled mathematical symbols like σ becoming σ or Σ , and completely broken document hierarchy where section headings are lost.

2.2 What Successful Conversion Looks Like

The goal is not perfect conversion—we acknowledge that perfection is aspirational at best. Rather, we aim for *useful* output: markdown that preserves:

1. **Document structure** — headings, subheadings, lists, tables in proper hierarchy
2. **Mathematical formulas** — as properly formatted LaTeX (σ_y , $M_b \leq \phi_b k_b M_s$) rather than garbled Unicode
3. **Referential integrity** — clause numbers, cross-references, appendices
4. **Semantic meaning** — the extracted text should be *readable* by a human and *parseable* by downstream tools

This methodology enables computational workflows that were previously impossible with scanned standards: automated formula lookup, cross-document analysis, and integration with calculation engines.

3. The Conversion Workflow

The conversion process consists of five sequential stages, each building on the previous:

Image Preparation → AI Extraction → Per-Page Output → Document Assembly → Quality Review

Stage 1: Image Preparation

The quality of extraction begins with the quality of input images. Poor scanning or inconsistent settings propagate errors through the entire pipeline.

Key considerations:

- **Resolution.** Scan at 300–400 DPI for optimal text recognition. Lower resolutions lose fine mathematical symbols; higher resolutions waste processing time without improving accuracy.
- **Color depth.** Grayscale is sufficient for most technical documents. Color adds file size without improving text recognition.
- **Page splitting.** Each page should be a separate image file. Multi-page PDFs must be split before extraction.

- **Cleanup.** Remove scan artifacts like shadows, stains, or paper discoloration that confuse the AI model's vision component.

Best practice: Use a consistent scanning profile across all pages of a document. Inconsistent settings cause variable extraction quality that makes downstream assembly difficult.

Stage 2: The Extraction Step

This is the core of the methodology: feeding prepared images to a multimodal AI model and receiving structured markdown output.

The general process:

1. **Select an appropriate model.** Look for models specifically fine-tuned for document understanding or OCR. Not all vision models handle technical content equally.
2. **Prepare the image.** Convert to a standard format (PNG recommended) and ensure it is not rotated or mirrored.
3. **Construct the prompt.** The prompt is arguably more important than the model choice—a well-crafted prompt can compensate for a weaker model.
4. **Submit in manageable batches.** Process 10–50 pages at a time to avoid rate limits and allow quality inspection per batch.
5. **Handle failures gracefully.** Models may timeout on dense pages. Resubmit failed pages with adjusted settings rather than abandoning the document.

The critical role of prompting. The prompt determines what kind of output the model produces. A generic prompt (“transcribe this image”) will produce variable results. A prescriptive prompt that explicitly instructs LaTeX formatting, heading hierarchy, and table preservation produces consistent, structured output.

The exact prompt varies by use case, but all effective prompts share these elements: - Explicit instruction to use LaTeX for math ($\$$. . $\$$ for inline, $\$ \$$. . $\$ \$$ for display) - Heading level guidance (h1 for main sections, h2 for subsections) - Table preservation instructions - Structure over prose (preserve formatting even if it looks unusual)

Stage 3: Per-Page Output

After extraction, each page exists as a separate markdown file. At this stage:

- **Verify completeness.** Ensure every page produced output. Missing pages indicate extraction failures that need resubmission.
- **Check formula rendering.** Spot-check mathematical formulas to ensure they use valid LaTeX syntax rather than Unicode symbols.
- **Validate structure.** Confirm heading levels match the document's actual hierarchy, not just visual size.

Stage 4: Document Assembly

Individual page markdowns must be merged into a single coherent document:

- **Sequential merge.** Join pages in their original order.
- **Page separator.** Insert a visible delimiter (e.g., --- Page N ---) between sections for traceability.
- **Table of contents.** If the source document had a TOC, ensure it is preserved or regenerated.
- **Appendix consolidation.** Pages labeled as appendices should remain at the end, not inserted at their original scan position.

Stage 5: Quality Review

The final step is human review to catch what automated processes miss:

- **Formula accuracy.** Do the extracted formulas match the source images? Common failures include sign errors, wrong variable names, and missing subscripts.
- **Section numbering.** Does clause 5.2.1 in the output match clause 5.2.1 in the source?
- **Table integrity.** Are table columns aligned correctly? Are merged cells preserved?
- **Cross-references.** Do “see Section X” references point to the correct section in the extracted document?

4. Preserving Mathematical Formulas

Mathematical notation is the defining challenge of technical document conversion. Unlike prose text, math has strict syntactic rules that must be preserved.

4.1 The LaTeX Advantage

LaTeX is the de facto standard for mathematical typesetting. Converting formulas to LaTeX syntax ($E = mc^2$, $\int_0^\infty e^{-x} dx$) provides:

- **Machine readability.** LaTeX can be parsed by mathematical libraries, rendering engines, and computation tools.
- **Universality.** LaTeX renders consistently across platforms and languages.
- **Precision.** Unicode math symbols ($\sum$, $\int$, $\leq$) lose semantic meaning; LaTeX preserves it.

4.2 Common Formula Types and Extraction Challenges

Formula Type	Example Challenge
Inline equations	Often split across lines in multi-column layouts
Displayed equations	Centered on their own line; easy to miss without explicit instructions
Subscripts/superscripts	Easily confused (a_i vs a^i)
Integral signs	Unicode $\int$ vs LaTeX $\int$
Greek letters	σ vs σ , θ vs θ
Operator precedence	Complex nested expressions

4.3 Model Selection for Math Extraction

Not all multimodal models handle math equally. When selecting a model:

- **Prefer models fine-tuned on mathematical content.** General vision-language models often produce poor math extraction.
- **Test with known formulas.** Before processing an entire document, extract a page with diverse formula types and verify output.
- **Consider parameter count.** Larger models (7B+ parameters) generally produce better math extraction than smaller ones.

5. Quality Considerations

5.1 Document Structure vs. Visual Appearance

AI models sometimes confuse visual appearance with semantic structure. A large centered title might be extracted as h1 when it is actually a section subtitle. To mitigate:

- **Provide context in the prompt.** Instruct the model to consider document hierarchy, not just font size.
- **Verify with source.** Cross-check extracted heading levels against the original document's TOC.
- **Post-process if needed.** Manual heading adjustment is often faster than perfect extraction on the first pass.

5.2 Handling Complex Layouts

Multi-column documents, footnotes, and sidebars present specific challenges:

- **Column ordering.** Models may read right column before left. Instruct the model to process left-to-right, top-to-bottom.
- **Footnotes.** Ensure footnotes are associated with their reference markers, not extracted as separate text.
- **Sidebars/callout boxes.** These should be flagged as distinct content blocks, not merged into main text.

5.3 Batch Processing Considerations

When processing documents in batches:

- **Use consistent prompts.** The same prompt for every batch ensures uniform output format.
- **Track batch boundaries.** Note where one batch ends and another begins for debugging.
- **Inspect the first page of each batch.** Catch systematic errors early rather than discovering them after processing 100 pages.

5.4 Batch Size Optimization

Optimal batch size depends on document complexity:

Document Type	Recommended Batch Size	Rationale
Text-heavy (minimal math)	50-100 pages	Faster processing, lower risk of errors
Mixed content (some formulas)	20-50 pages	Balance between speed and quality
Math-heavy (dense formulas)	10-20 pages	Allows careful formula extraction per page
Complex tables	10-20 pages	Table structure requires more model attention

Quality gate strategy: Process the first batch of 10 pages and inspect thoroughly. If quality is acceptable, increase to your optimal batch size. If quality is poor, refine the prompt before continuing.

5.5 Cost and Hardware Considerations

Cloud API models (GPT-4o, Claude, Gemini): - Cost: Typically \$5-30 per 100 pages depending on model and resolution - Advantage: No hardware required, consistent quality - Best for:

Documents under 200 pages, one-time extraction projects

Self-hosted models (Qwen-VL, LLaVA): - Hardware: Minimum 8GB VRAM recommended for 7B parameter models - Cost: Electricity + GPU depreciation; essentially free after hardware investment - Best for: Large document archives, repeated extraction workflows, privacy-sensitive content

Hybrid approach: Use local models for routine extraction and cloud APIs as fallback for pages that fail local extraction.

6. Post-Processing and Validation

6.1 Automated Validation

Several checks can be automated before human review:

- **LaTeX syntax validation.** Parse extracted formulas with a LaTeX linter to catch malformed expressions.
- **Heading hierarchy check.** Ensure $h1 > h2 > h3$ nesting is consistent (no skipping levels).
- **Table detection.** Verify that recognized tables have matching opening and closing delimiters.
- **Page count verification.** Extracted page count should match source document page count.

6.2 Manual Review Priorities

When reviewing extracted documents, prioritize:

1. **First 10 pages.** Catch systematic prompt issues early.
2. **Pages with heavy math.** These are highest value and most error-prone.
3. **Cross-reference points.** Verify that “see Section X” references resolve correctly.
4. **Appendices.** Often overlooked by AI models due to their non-linear content.

7. Tool and Model Agnosticism

This methodology is intentionally tool-agnostic. The specific tools you use are less important than following the correct sequence:

Model categories that work: - Multimodal vision-language models (LLaVA, Qwen-VL, Claude Computer Use) - Specialized OCR engines with math support (Azure Form Recognizer, AWS Textract) - General-purpose LLMs with image input (GPT-4o, Gemini Pro)

Implementation approaches that work: - Cloud API services (paid per page) - Local self-hosted solutions (free but require GPU) - Hybrid approaches (local preprocessing, cloud extraction)

The common thread is not the tool but the workflow: prepare images carefully, craft precise prompts, verify output quality, and assemble systematically.

8. Limitations and Realistic Expectations

8.1 What This Methodology Cannot Do

- **Perfect extraction.** Some formulas and symbols will always be ambiguous, even to human readers.
- **Preserve exact formatting.** Hand-drawn annotations, custom diagrams, and proprietary notation will be approximated.

- **Replace human review.** The output is useful but not production-ready without verification.

8.2 When Manual Intervention is Necessary

- Models struggle with heavily stained or faded pages
- Extremely small font sizes (< 6pt) lose recognition accuracy
- Handwritten notes in margins are often misattributed to nearby text
- Complex multi-level equations sometimes collapse into single-line approximations

In these cases, manual correction of the extracted markdown is expected and normal.

9. Conclusion

Converting technical documents to machine-readable markdown with mathematical fidelity is achievable with the right methodology. The key insights are:

1. **Image quality matters more than model quality.** A good model with poor images produces worse results than a mediocre model with excellent scans.
2. **Prompt design is the most important skill.** A well-crafted prompt compensates for model limitations; a poor prompt wastes even the best models.
3. **Batch processing with quality gates** prevents catastrophic failure and enables early correction.
4. **Human review remains essential.** AI extraction is a powerful assistant but not a replacement for expert verification.

The workflow—prepare, extract, verify, assemble, review—is tool-agnostic and model-agnostic. It applies equally to converting a 20-page research paper or a 500-page engineering standard. The principles are universal; the specific implementation details vary.

What this methodology enables is transformative: scanned technical documents become searchable, computable, and cross-referencable. Engineers can find every formula involving stress analysis across a 400-page standard. Researchers can grep for specific design criteria in seconds rather than hours. Tools can parse extracted formulas for automated calculation.

The barrier is no longer technology—it is methodology. Follow the workflow, respect the quality gates, and invest in prompt refinement. The output will be useful, accurate, and transformative for your workflow.

This document describes Version 1.0 of the Technical Document Conversion Methodology. Subsequent versions may address limitations identified through production use.